

NapCart

Product Requirements Document (PRD)

Version: v1.0 Date: 2026-05-28 Status: Draft Author: Codex with Naptime AI

1. Executive Summary

1.1 Product Definition

NapCart is a reusable ordering and delivery-management platform for small and growing restaurants that currently depend on WhatsApp and phone calls to handle delivery orders manually.

The product is not a full restaurant website/content platform. It is a structured ordering system that can:

- power a simple branded storefront when a restaurant needs a new site
- work as a reusable ordering backend and admin system
- later integrate with an existing restaurant website through APIs

1.2 Core Problem

Small restaurants often receive delivery orders through unstructured chats. Staff must manually ask for:

- items
- quantities
- address
- branch preference
- phone number
- delivery notes

This causes delays, incomplete orders, avoidable mistakes, and weak operational visibility.

1.3 Product Goal

Move order capture from manual WhatsApp chats to a structured online ordering flow, while keeping restaurant staff operationally centered on WhatsApp.

1.4 Product Vision

```
Customer Website -> Structured Order -> Branch WhatsApp -> Staff Confirm/Cancel
| | | |
| | | |
+-----> Database / Admin Dashboard / Reporting <-----+
```

1.5 MVP Outcome

The MVP should let a customer place a correct order online, route that order to the correct restaurant branch on WhatsApp, allow staff to confirm or cancel from WhatsApp, notify the customer of the result, and store all operational data in the backend for management visibility.

2. Business Context

2.1 Business Goals

- Reduce time spent manually taking orders
- Reduce order mistakes caused by incomplete chats
- Improve professionalism of restaurant operations
- Create a reusable product for multiple restaurant clients
- Keep the architecture flexible for future monetization and scale

2.2 Launch Strategy

- Launch with one real restaurant first
- Keep architecture multi-tenant-ready from day one
- Use a single codebase
- Use a standalone storefront + admin for MVP
- Keep backend API-first for future website integrations

2.3 Success Criteria

The MVP will be considered successful if it achieves the following:

- customers can place valid delivery and pickup orders without staff chat assistance
- orders reach the correct branch WhatsApp in a clean, structured format
- staff can confirm or cancel the order through WhatsApp
- customer receives confirmation or cancellation message
- management can review orders, customers, menu data, and basic analytics in the dashboard

3. Users and Stakeholders

3.1 Primary Users

- Restaurant owners
- Restaurant managers
- Branch staff receiving orders on WhatsApp
- Customers placing delivery or pickup orders

3.2 Secondary Users

- Delivery coordinators
- Future branch admins
- Future riders

3.3 User Roles in MVP

Customer

- > browses menu
- > chooses branch
- > places order

Branch Staff (WhatsApp-first)

- > receives order on branch WhatsApp
- > confirms or cancels order

Restaurant Admin / Owner

- > uses dashboard for visibility, menu management, settings, and analytics

4. Product Principles

- WhatsApp-first restaurant operations
- Structured data capture before human intervention
- Dashboard as system of record
- Minimal operational behavior change for restaurant staff
- Multi-tenant-ready design from the start
- Fast MVP delivery without overbuilding

5. Scope Definition

5.1 MVP Scope Summary

IN MVP

- guest checkout
- English only
- delivery + pickup
- multi-branch support
- manual branch selection
- branch-level WhatsApp routing
- menu management in internal admin
- product variations and add-ons
- restaurant logo and product image uploads
- branch and restaurant availability controls
- delivery zones and delivery fees
- configurable minimum order amount
- WhatsApp-based Confirm / Cancel actions
- customer confirmation / cancellation message
- one main admin account per restaurant
- basic analytics

OUT OF MVP

- customer accounts/login
- OTP verification
- coupons/discount codes
- scheduled orders
- customer order history / reorder flow
- tax/VAT handling
- advanced analytics
- CSV/data export
- advanced order lifecycle tracking
- role-based admin permissions
- multilingual support
- rider management
- online payments
- branch auto-assignment
- reminder automation for unresponded orders

5.2 Priority Matrix

P0 Must Have

- structured storefront ordering flow
- branch selection
- cart and checkout
- delivery and pickup options
- delivery zone and fee rules
- internal menu management
- WhatsApp branch order routing
- WhatsApp Confirm / Cancel workflow
- backend order storage
- customer notification after branch decision
- admin dashboard core modules

P1 Should Have Soon After MVP

- richer analytics
- branch auto-assignment
- exports
- customer notifications for more statuses
- improved anti-spam/validation checks

P2 Future

- online payments
- rider management
- coupons
- scheduled orders
- customer accounts
- loyalty and CRM features

6. Product Visualization

6.1 System Scope Map

```
+-----+
| Restaurant Admin |
| Dashboard |
+-----+-----+
|
v
+-----+ +-----+ +-----+ +-----+
| Customer | ----> | Ordering / Checkout Platform | ----> | Branch WhatsApp |
| Storefront | | Menu, Cart, Routing, Orders | | Staff Confirm/Cancel |
+-----+ +-----+ +-----+ +-----+
| | |
| v v
| +-----+ +-----+ +-----+
+-----> | Database / Audit Log | <----- | Customer Notification |
+-----+ +-----+
```

6.2 Order Lifecycle in MVP

```
Placed
|
v
Pending Confirmation
| \
| \
| +--> Cancelled
|
+-----> Confirmed
```

6.3 Branch Routing Visualization

```
Customer chooses Branch A
|
v
Order.branch_id = Branch A
|
v
System loads Branch A WhatsApp destination
|
v
Order sent to Branch A WhatsApp
```

7. User Journeys

7.1 Customer Journey

```
Landing -> Branch Selection -> Menu Browse -> Product Selection
-> Cart -> Checkout -> Place Order -> Await Confirmation
-> Receive Confirm/Cancel Message
```

7.2 Branch Staff Journey

```
Receive structured order on WhatsApp
-> Review order
-> Tap Confirm or Cancel
-> Backend status updates
-> Customer notification is sent
```

7.3 Admin Journey

```
Login -> Review dashboard metrics -> Manage menu/settings/branches
-> Review order history -> Monitor pending confirmations
```

8. Functional Requirements

8.1 Storefront

The storefront must support:

- branded restaurant presentation
- manual branch selection
- branch-aware ordering
- menu categories
- product images

- product descriptions
- base pricing
- variations
- add-ons
- cart management
- delivery or pickup selection
- free-text address entry for delivery
- customer details entry
- order notes
- order confirmation screen

Acceptance Criteria

- customer can complete an order without creating an account
- selected branch is attached to the order
- checkout validates required fields before order creation
- unavailable products cannot be ordered
- closed restaurant or branch must block ordering clearly

8.2 Menu Management

The internal admin menu system must support:

- categories
- products
- product descriptions
- base price
- product images
- product active/inactive state
- variations
- add-on groups
- add-ons
- per-product availability toggle

Acceptance Criteria

- admin can create and update products without code changes
- menu changes affect future orders only
- variations and add-ons are reflected in cart and order summaries

8.3 Branch Management

The system must support:

- one or more branches per restaurant
- manual branch selection by the customer
- branch-specific WhatsApp routing
- branch-aware order storage
- branch-level operational settings later

Acceptance Criteria

- each order is attached to one branch
- the system routes WhatsApp message to the correct branch destination
- restaurants with one shared WhatsApp number can still use branch labels in the order message

8.4 Delivery and Pickup Rules

The system must support:

- delivery + pickup fulfillment types
- configurable delivery zones
- configurable delivery charges
- configurable minimum order amount
- restaurant-defined delivery coverage rules

Acceptance Criteria

- delivery fee is calculated according to restaurant settings
- delivery can be blocked when the area is outside supported rules
- pickup orders do not require delivery fee

8.5 Order Capture

When a customer places an order, the system must:

- validate the payload
- save order header and line items
- save snapshot data for product names, prices, selected options, and add-ons
- save customer details
- set order status to Pending Confirmation
- send a WhatsApp order message to the selected branch

Acceptance Criteria

- order exists in database before outbound WhatsApp action
- system retains accurate order details even if menu changes later
- order reference is unique and easy for staff to identify

8.6 WhatsApp Operations Workflow

The WhatsApp workflow must support:

- sending new order details to the branch
- showing two clear actions: Confirm and Cancel
- receiving the staff response
- updating backend status
- sending resulting customer message

MVP Rule

- no advanced branch-side status flow beyond Confirm / Cancel
- if staff does not respond, order remains Pending Confirmation
- no automatic reminder or re-send behavior in MVP

Acceptance Criteria

- branch staff can act without opening the admin dashboard

- customer gets the correct result message after staff action
- status change is logged in backend

8.7 Customer Records

The backend must maintain internal customer records even though the customer experience is guest checkout only.

Required customer tracking:

- internal customer ID
- normalized phone number
- name
- order count
- first order date
- last order date
- future-ready address history

Matching Rule

- internal customer ID is the primary key
- normalized phone number per restaurant is the main matching key
- customer name is display data only

8.8 Admin Dashboard

The dashboard must support:

- one main admin login per restaurant
- order list
- order detail view
- pending confirmation visibility
- customer list
- menu management
- branch configuration
- delivery settings
- restaurant settings
- availability controls
- WhatsApp configuration
- basic analytics

Basic Analytics in MVP

- total orders
- confirmed orders
- cancelled orders
- pending confirmation orders
- total sales
- customer count

8.9 Availability and Operational Controls

The system must let restaurants:

- mark restaurant closed now
- define opening hours
- temporarily stop accepting orders
- mark individual products unavailable

Acceptance Criteria

- storefront must clearly communicate when ordering is unavailable
- unavailable products must not be orderable

8.10 Branding

Each restaurant must be able to manage:

- restaurant name
- logo
- contact details
- basic branding presentation

The architecture must also remain ready for future custom domains.

9. Non-Functional Requirements

9.1 Performance

- storefront pages should feel fast on mobile-first usage
- checkout should remain lightweight and direct
- core order creation API should be responsive under normal restaurant traffic

9.2 Reliability

- order must be saved before outbound messaging
- WhatsApp message attempts must be logged
- failed messaging attempts must be visible for later retry tooling

9.3 Security

- admin access must require authentication
- WhatsApp credentials must never be exposed client-side
- sensitive secrets must be environment-based
- checkout endpoints should be rate-limited
- inputs must be validated server-side

9.4 Scalability

- schema must be restaurant-aware from day one
- branch routing must be configurable
- backend must stay API-first for future integration with existing websites

9.5 Maintainability

- menu management must not require developer edits
- branch onboarding should be mostly configuration-driven
- WhatsApp provider logic must sit behind an adapter/service layer

10. Data Requirements

10.1 Core Entities

```
Restaurant
-> Branch
-> Admin User
-> Branding / Settings

Restaurant
-> Category
-> Product
-> Product Variant
-> Add-on Group
-> Add-on

Restaurant
-> Customer
-> Customer Address
-> Order
-> Order Item
-> Order Item Add-on
-> WhatsApp Message Log
```

10.2 Important Data Rules

- all operational records must reference restaurant_id
- all orders must reference branch_id
- order items must store snapshot values
- customer matching must rely on normalized phone number, not name
- address must be stored as free text with optional future lat/lng fields

11. Status Model

11.1 Order Statuses in MVP

- Pending Confirmation
- Confirmed
- Cancelled

11.2 Deferred Statuses

The following are intentionally not part of the operational MVP workflow:

- Preparing
- Dispatched
- Delivered

Reason:

- staff remains WhatsApp-first
- no reliable real-time signal exists for later statuses in MVP
- forcing these states would create false precision and operational friction

12. WhatsApp Integration Requirements

12.1 Provider Strategy

- standard provider: Meta WhatsApp Cloud API
- development mode: mock/testing mode supported
- architecture: provider adapter/service layer

12.2 Branch Routing Model

- each branch can map to its own WhatsApp destination
- branches with different receiving numbers require separate branch mappings
- restaurants with one shared number can still receive branch-labeled orders

12.3 Customer Notification Scope

Customer notifications in MVP are limited to:

- order confirmed
- order cancelled

No other customer-facing status notifications are required in MVP.

13. Out of Scope for MVP

- online payments
- customer accounts
- OTP phone verification
- customer order history and reorder experience
- multilingual support
- scheduled orders
- coupons and discount codes
- tax/VAT configuration
- preparing/dispatched/delivered operational tracking
- rider management
- role-based staff permissions
- export tools
- reminder automation
- branch auto-assignment
- large content/SEO website management system

14. Risks and Mitigations

Risk 1: WhatsApp onboarding/configuration complexity

Mitigation: provider adapter design + mock mode + per-branch config model

Risk 2: Branch staff ignores backend completely

Mitigation: make Confirm/Cancel workflow operational through WhatsApp

Risk 3: Delivery-zone logic varies across restaurants

Mitigation: keep delivery rules configurable in restaurant settings

Risk 4: Menu modeling becomes inconsistent

Mitigation: build internal menu CMS with product, variant, and add-on structure

Risk 5: Existing restaurant websites need integration later

Mitigation: keep backend API-first and avoid hard-coupling frontend assumptions

15. Acceptance Criteria

The MVP is acceptable when all of the following are true:

- a restaurant can configure branches, menu, branding, availability, and delivery rules
- a customer can select a branch and place an order through guest checkout
- the system stores the order and customer data correctly
- the order is routed to the correct branch WhatsApp
- branch staff can confirm or cancel via WhatsApp
- customer receives confirmation or cancellation message
- admin dashboard shows core order and customer visibility
- basic analytics are available

16. Delivery Phases After PRD

Phase A: ERD and data design

Phase B: architecture and API specification

Phase C: storefront and checkout implementation

Phase D: menu and settings dashboard

Phase E: WhatsApp confirm/cancel workflow

Phase F: testing, pilot launch, iteration

17. Appendix: Finalized MVP Decisions

Launch model: one real restaurant first, multi-tenant-ready architecture
Frontend model: standalone storefront + admin, API-first backend
Checkout: guest only
Language: English only
Payments: cash on delivery / cash on pickup only
Fulfillment: delivery + pickup
Branches: supported
Branch selection: manual
Delivery rules: configurable zones, fees, minimum order amount
Menu model: internal admin-driven menu system
Variations/add-ons: included
Image uploads: product images + restaurant logo
Availability controls: included
Customer history UI: excluded
Internal customer records: included
Address model: free text now, future-ready coordinates
Tax: deferred
Coupons: deferred
Scheduled orders: deferred
Analytics: basic only
Notifications: restaurant WhatsApp first, customer confirm/cancel only
Order workflow: Pending Confirmation -> Confirmed / Cancelled
Reminder automation: excluded
Admin users: one main admin per restaurant